A Comparative Evaluation of Bio-Inspired Hyperparameter Optimization Algorithms for Software Defect Density Prediction

Juan Murillo Morera[1 [0000−0003−0871−8263]], Rubén Fuentes Fernández[2 [0000−0001−6876−0979]], Andrés Calvo Vargas[1 [0009−0000−3615−0615]], Alessandro Cambronero Delgado[1 [0009−0004−1489−6175]], Janio Jadán-Guerrero[3 [0000−0002−3616−2074]], and Susana Alexandra Arias[4 [0000−0002−5337−3156]]

[1] School of Informatics and Computing, National University of Costa Rica, Costa Rica
[2] Institute of Knowledge Technology & Research Group on Agent-based, Social & Interdisciplinary Applications, Complutense University of Madrid, Madrid, Spain
[3] Research Center for Human Sciences and Education (CICHE), Indoamérica Technological University, Quito, Ecuador
[4] International University of La Rioja, Spain
juan.murillo.morera@una.cr

Abstract. Software defect density prediction remains a challenging problem in software quality assurance. Although bio-inspired optimization techniques have been widely adopted for hyperparameter tuning, limited empirical evidence exists regarding their comparative effectiveness in defect density prediction environments. This study presents a comparative evaluation of four bio-inspired optimization algorithms–Genetic Algorithm (GA), Particle Swarm Optimization (PSO), Artificial Bee Colony (ABC), and Composite Differential Evolution Algorithm (CDEA)–within a previously validated software defect density prediction framework. The empirical study comprises 550 experimental executions involving ten PROMISE datasets, eleven regression algorithms, and a common feature selection strategy. Results indicate that all evaluated optimization algorithms achieve statistically equivalent predictive performance in terms of RMSE. However, significant differences were identified in computational efficiency, with GA and CDEA obtaining superior execution-time rankings. Friedman statistical analysis confirms that the choice of optimizer has a greater impact on computational cost than on predictive accuracy. These findings provide empirical evidence supporting the use of bio-inspired optimization techniques as efficient mechanisms for hyperparameter configuration in software defect density prediction.

Keywords: Software defect prediction · Defect density · Hyperparameter optimization · Bio-inspired algorithms · Regression models · Empirical software engineering

1 Introduction

Software reliability is a critical concern in software engineering [19]. As software systems become increasingly complex, organizations require effective mechanisms for identifying defect-prone components, estimating defect density, and allocating testing resources efficiently. Defect prediction models usually rely on software metrics extracted from source code, development history, or design artifacts, and have been extensively studied in empirical software engineering [12, 14, 21, 23].

Software fault prediction research can be broadly organized into three categories [22]: estimating the number of defects or defect density in a system, discovering associations among defects, and classifying software modules as defect-prone or not. This paper focuses on regression-based defect density prediction, where the objective is to estimate the number of defects associated with a software module. This task remains difficult because software defect datasets often contain noise, skewed distributions, missing values, outliers, and heterogeneous metric profiles. Public repositories such as NASA-MDP and PROMISE have therefore required careful preprocessing in previous studies [13, 20].

Hyperparameter optimization has become an important step for improving the performance of machine learning models in software defect prediction. A typical pipeline includes data preprocessing, feature selection, model training, and hyperparameter tuning. However, different optimization strategies may lead to different computational costs even when their predictive performance is similar. This raises a relevant empirical question: when several bio-inspired algorithms are available, which one provides the best trade-off between predictive accuracy and execution time?

This paper extends the framework previously validated by Murillo et al. [16]. Unlike that earlier work, which focused on validating a hyperparameter optimization framework using a Genetic Algorithm and an exhaustive baseline, the present study evaluates four bio-inspired optimization algorithms under identical experimental conditions: GA, PSO, ABC, and CDEA. The main contribution is therefore not the proposal of a new framework, but the comparative empirical evaluation of alternative bio-inspired optimization strategies for software defect density prediction.

Following the Goal-Question-Metric (GQM) paradigm [2], the objective of this research is stated as follows. *Analyze* four bio-inspired hyperparameter optimization algorithms, *for the purpose of* comparing predictive performance and computational efficiency, *with respect to* RMSE and execution time, *from the point of view of* researchers and software engineering practitioners, *in the context of* regression-based software defect density prediction using PROMISE datasets.

The remainder of this paper is organized as follows. Section 2 presents the research questions. Section 3 summarizes related work. Section 4 describes the empirical study. Section 5 presents the results and statistical analysis. Section 6 discusses threats to validity. Section 7 concludes the paper and outlines future work.

2 Research Questions

This study is guided by four research questions:

- **RQ1:** Among the evaluated bio-inspired optimization algorithms (GA, PSO, ABC, and CDEA), which hyperparameter configurations yielded the best predictive performance as measured by RMSE across the ten PROMISE datasets?
- **RQ2:** Which machine learning algorithms demonstrated the highest predictive accuracy in terms of RMSE when optimized under the evaluated hyperparameter search strategies?
- **RQ3:** Are there statistically significant differences in execution time among the bio-inspired optimization algorithms?
- **RQ4:** Are there statistically significant differences in RMSE among the evaluated bio-inspired optimization algorithms?

3 Related Work

3.1 Software Defect Prediction

Software defect prediction has been widely investigated using both classification and regression approaches. Menzies et al. [13] showed the usefulness of static code attributes for learning defect predictors, while Song et al. [22] proposed a general software defect-proneness prediction framework. Lessmann et al. [11] conducted a large-scale benchmark of classification models, highlighting the importance of rigorous empirical comparison. Malhotra [12] later provided a comparative analysis of statistical and machine learning methods for predicting faulty modules.

Several studies have also addressed data quality, feature selection, imbalance, and noisy software metrics. Shepperd et al. [20] emphasized the need to handle quality issues in NASA defect datasets. Huda et al. [6] proposed a framework combining SVM and ANN with metric selection, while Bashir et al. [1] integrated feature selection, data balancing, and noise filtering. More recently, Hesamolhokama et al. [5] introduced a reinforcement-learning-based ensemble framework for software defect prediction.

3.2 Hyperparameter Optimization and Bio-Inspired Algorithms

Hyperparameter optimization is a central component of modern machine learning pipelines. Bischl et al. [3] discuss foundations, algorithms, best practices, and open challenges in this area. In software engineering, the number of possible configurations can increase rapidly when preprocessing methods, feature selectors, learning algorithms, and model-specific parameters are combined.

Bio-inspired algorithms provide an alternative to deterministic search strategies because they can explore large and complex spaces without evaluating every possible configuration. Genetic Algorithms have been previously used in defect prediction frameworks by Murillo et al. [15, 17, 18]. PSO has been widely used

for continuous optimization problems [9], ABC is inspired by the foraging behavior of honey bees [8], and Differential Evolution variants such as CDEA have shown strong performance in numerical optimization [24]. A broader review of bio-inspired optimization algorithms is provided by Jakšić et al. [7].

3.3 Research Gap

The previous framework by Murillo et al. [16] validated bio-inspired hyperparameter optimization for software defect density prediction mainly through the use of a Genetic Algorithm. However, limited empirical evidence exists on whether alternative bio-inspired optimizers behave differently when applied under the same defect density prediction pipeline. This paper addresses that gap by comparing GA, PSO, ABC, and CDEA across the same datasets, regression algorithms, feature selection method, and evaluation protocol.

4 Empirical Study

4.1 Datasets

The empirical evaluation used ten datasets from the PROMISE Software Engineering Repository [10]: *ant-1*, *poi-2*, *camel-1*, *tomcat*, *ivy-2*, *velocity-1*, *jedit-4*, *xalan-2*, *log4j-1*, and *xerces-1*. These datasets contain object-oriented software metrics, such as CBO, DIT, and LCOM, and the target variable corresponds to the number of defects per software module. The datasets were used as regression tasks for defect density prediction.

4.2 Learning Algorithms and Hyperparameters

The study considered eleven regression algorithms: Linear Regression, Decision Tree Regressor, Random Forest Regressor, Gradient Boosting Regressor, Lasso, Ridge, ElasticNet, K-Nearest Neighbors Regressor, Support Vector Regressor, Multi-Layer Perceptron Regressor, and XGBRegressor. A single feature selection strategy was applied: `SelectFromModel` with `LassoCV`, which selects attributes based on the L1-regularization path. Table 1 summarizes the hyperparameter search space.

4.3 Optimization Algorithms

Four bio-inspired hyperparameter optimization algorithms were evaluated:

- **Genetic Algorithm (GA):** an evolutionary method based on selection, crossover, mutation, and elitism [15, 18].
- **Particle Swarm Optimization (PSO):** a swarm intelligence technique in which particles update their positions based on individual and collective experience [9].

Table 1. Summary of learning algorithms and hyperparameters.

Model	Hyperparameters
Linear Regression	`copy_X`, `fit_intercept`, `positive`
Decision Tree	`max_depth`, `min_samples_split`, `min_samples_leaf`, `random_state`
Random Forest	`n_estimators`, `max_depth`, `min_samples_split`, `random_state`
Gradient Boosting	`n_estimators`, `learning_rate`, `max_depth`, `subsample`
Lasso	`alpha`, `fit_intercept`, `max_iter`, `tol`, `selection`
Ridge	`alpha`, `fit_intercept`, `tol`, `solver`
ElasticNet	`alpha`, `l1_ratio`, `fit_intercept`
KNN Regressor	`n_neighbors`, `weights`, `algorithm`
SVR	`kernel`, `C`, `epsilon`, `gamma`
MLP Regressor	`hidden_layer_sizes`, `activation`, `alpha`, `learning_rate_init`
XGBRegressor	`learning_rate`, `n_estimators`, `max_depth`, `subsample`, `colsample_bytree`

- **Artificial Bee Colony (ABC):** a swarm-based algorithm that simulates the foraging behavior of employed, onlooker, and scout bees [8].
- **Composite Differential Evolution Algorithm (CDEA):** a Differential Evolution variant that combines multiple trial vector generation strategies and control parameters [24].

4.4 Experimental Protocol

The experimental design follows the previous defect density prediction framework [16], but extends the optimization layer to four bio-inspired algorithms. The experimental matrix was generated from ten datasets, eleven learning algorithms, one attribute selector, and four bio-inspired optimization strategies. The resulting experimental database contains 550 execution records after considering optimizer-specific runs and repeated evaluations. This design enables the comparison of predictive performance and computational efficiency under consistent preprocessing, feature selection, and evaluation conditions.

For each dataset, 70% of the data were randomly selected for training and 30% were reserved for testing. This process was repeated ten times using different random partitions. Missing values were removed, the target variable was transformed using `log(defects + 1)`, and features were standardized using `StandardScaler`. Each model was evaluated using 10-fold cross-validation within the training subset and then assessed on the held-out test subset. Performance was measured using RMSE and R^2, while execution time was recorded to evaluate computational efficiency.

Given the comparison of more than two optimization algorithms, the Friedman test was used as a non-parametric omnibus test. A significance level of $\alpha = 0.05$ was adopted. When statistically significant differences were detected,

post-hoc analysis was considered to identify pairwise differences among optimizers.

5 Results and Analysis

This section addresses the research questions using the experimental results obtained from ten PROMISE datasets, eleven regression algorithms, one attribute selection method, and four bio-inspired optimization algorithms. The analysis focuses on two complementary dimensions: predictive performance, measured mainly through RMSE, and computational efficiency, measured through execution time.

5.1 RQ1: Best Hyperparameter Configurations

The experimental results indicate that all evaluated bio-inspired optimization algorithms were capable of identifying competitive hyperparameter configurations across the ten PROMISE datasets. The best overall predictive performance was obtained on the *poi-2* dataset using the Lasso regressor, with an RMSE of approximately 0.2219. This result was independently identified by multiple optimization strategies, suggesting the existence of stable high-quality regions within the hyperparameter search space.

This finding is relevant because software defect density datasets commonly exhibit heterogeneous distributions, redundant metrics, and varying defect patterns across projects. Despite these characteristics, GA, PSO, ABC, and CDEA converged toward solutions with highly similar predictive quality. Therefore, the evaluated search strategies demonstrated sufficient exploration capability to locate near-optimal configurations under the same preprocessing, feature selection, and evaluation conditions.

The results also show that the best configuration is not determined exclusively by the optimizer. Instead, predictive performance emerges from the interaction among dataset characteristics, selected learning algorithm, feature selection, and hyperparameter values. Consequently, hyperparameter optimization remains a dataset-dependent process in software defect density prediction. The evidence obtained in this study suggests that bio-inspired optimization provides a robust mechanism for identifying high-quality configurations without relying on a single search strategy.

5.2 RQ2: Best Predictive Models

Across datasets and optimization strategies, regularized linear models achieved the lowest average prediction errors. Ridge Regression, Linear Regression, and ElasticNet obtained the best average RMSE values. These models showed stable behavior across the evaluated optimization algorithms and maintained competitive performance even when compared with more complex learners.

A possible explanation for this behavior is related to the nature of software metrics datasets. PROMISE datasets often contain correlated predictors, limited sample sizes, and noisy metric distributions. Under these conditions, regularization can reduce variance, mitigate overfitting, and improve generalization. The performance of Ridge and ElasticNet suggests that controlling model complexity may be more beneficial than increasing representational capacity.

Ensemble methods such as Random Forest, Gradient Boosting, and XGBoost also produced competitive results, particularly for datasets with more complex nonlinear patterns. However, their average predictive performance was slightly inferior to that obtained by the strongest regularized linear models. This result indicates that more complex learning algorithms do not necessarily guarantee better defect density prediction, especially when the available data are noisy, skewed, or relatively small.

5.3 RQ3: Execution-Time Differences

Computational efficiency is a critical factor in hyperparameter optimization because each candidate configuration may require repeated model training, validation, and testing. To compare execution time among the evaluated optimizers, the Friedman test was applied to the runtime values obtained across the experimental configurations. Table 2 summarizes the statistical results for both execution time and RMSE.

Table 2. Friedman test results for execution time and RMSE.

Statistic	**Value**
RQ3 execution time: χ_F^2	226.59
RQ3 execution time: p-value	7.57×10^{-49}
RQ4 RMSE: χ_F^2	2.974
RQ4 RMSE: p-value	0.3957
Significance level	0.05

For execution time, the null hypothesis was rejected because the p-value was lower than 0.05. Therefore, statistically significant differences exist among the execution times of the evaluated bio-inspired algorithms. Table 3 presents the Friedman ranking for execution time.

The ranking shows that GA achieved the best computational efficiency, followed by CDEA and PSO. ABC obtained the worst execution-time ranking, indicating a higher computational cost under the evaluated configuration space. This behavior can be explained by the different search mechanisms used by each optimizer. GA applies relatively simple evolutionary operators, whereas ABC requires additional interactions among employed bees, onlooker bees, and scout bees, increasing the number of operations required during the search process.

Table 3. Friedman ranking for execution time. Lower rank indicates better computational efficiency.

Optimizer	Rank
GA	1.48
CDEA	2.23
PSO	2.30
ABC	3.98

These results have direct methodological implications. When predictive performance is statistically equivalent, runtime becomes a decisive criterion for optimizer selection. Faster optimization enables more iterations, broader sensitivity analyses, and more frequent model retraining, which are especially important in software engineering environments where defect prediction models may need to be updated as new project data become available.

5.4 RQ4: Predictive Performance Differences

To determine whether statistically significant differences existed in predictive performance, the Friedman test was applied to RMSE values obtained by GA, PSO, ABC, and CDEA. As shown in Table 2, the RMSE p-value was 0.3957. Therefore, the null hypothesis could not be rejected, indicating that no statistically significant differences were found among the predictive performances of the evaluated optimizers.

Table 4. Summary of predictive performance and execution time.

Optimizer	Mean RMSE	Mean Time (s)	Time Rank	RMSE Rank
GA	0.382104	1.121	1.48	1
ABC	0.382449	2.699	3.98	2
PSO	0.382556	1.242	2.30	3
CDEA	0.382825	1.296	2.23	4

Although small numerical differences were observed, all algorithms converged toward solutions with practically equivalent predictive quality. This result is important because GA, PSO, ABC, and CDEA use different exploration and exploitation mechanisms. The absence of significant RMSE differences suggests that the hyperparameter search spaces associated with software defect density prediction contain multiple regions of comparable quality.

The findings also strengthen the reproducibility of the framework. If several independent optimizers produce statistically equivalent predictive results under identical experimental conditions, practitioners can select an optimizer accord-

ing to secondary but relevant criteria, such as execution time, implementation complexity, parameter sensitivity, or available computational resources.

5.5 Discussion

The results obtained in this study provide several insights into the role of bio-inspired optimization in software defect density prediction. Although GA, PSO, ABC, and CDEA employ different search mechanisms, all algorithms converged toward solutions exhibiting statistically equivalent predictive performance. This observation indicates that computational efficiency becomes a more relevant differentiating factor than predictive accuracy.

The study also shows the importance of evaluating optimization algorithms under identical conditions. Previous comparisons are often difficult to interpret because they vary datasets, preprocessing procedures, learning algorithms, or validation protocols. In contrast, this study isolates the optimization layer and evaluates all optimizers within the same experimental environment. Consequently, the observed runtime differences can be attributed primarily to the optimization algorithms themselves.

5.6 Summary of Findings

Table 5 summarizes the answers to the research questions.

Table 5. Answers to the research questions.

RQ	Focus	Main finding
RQ1	Best configurations	All bio-inspired algorithms identified competitive configurations. The best RMSE was approximately 0.2219 on *poi-2* using Lasso.
RQ2	Best predictive models	Ridge Regression, Linear Regression, and ElasticNet obtained the lowest average RMSE values.
RQ3	Execution time	Significant differences were found. GA achieved the best computational ranking, whereas ABC required the highest execution time.
RQ4	RMSE differences	No statistically significant RMSE differences were found among GA, PSO, ABC, and CDEA.

6 Threats to Validity

Construct validity. This study uses RMSE and R^2 as evaluation metrics, which are commonly adopted for regression-based defect prediction. Nevertheless, additional measures such as MAE or relative error could provide complementary evidence in future studies.

Internal validity. The experimental protocol used repeated randomized partitions, standardized preprocessing, and a common feature selection method across all algorithms. However, implementation-specific parameters of each optimizer may influence execution time and convergence behavior.

Conclusion validity. Since more than two optimizers were compared, the Friedman test was used instead of pairwise-only tests. A significance level of $\alpha = 0.05$ was adopted. The results support the conclusion that runtime differs significantly across optimizers, while RMSE does not.

External validity. The evaluation was conducted using ten PROMISE datasets. Although these datasets are widely used in software defect prediction, additional validation using industrial datasets, larger repositories, and different programming languages is required before generalizing the results to all software development contexts.

7 Conclusions and Future Work

This study presented a comparative evaluation of four bio-inspired hyperparameter optimization algorithms–GA, PSO, ABC, and CDEA–for software defect density prediction. The empirical evaluation involved ten PROMISE datasets, eleven regression algorithms, a common feature selection mechanism, and 550 experimental executions.

The results showed that all evaluated optimizers achieved statistically equivalent predictive performance in terms of RMSE. However, significant differences were observed in execution time. GA obtained the best overall computational ranking, followed by CDEA and PSO, whereas ABC exhibited the highest computational cost.

These findings suggest that the selection of a bio-inspired optimizer should be guided primarily by computational efficiency rather than predictive performance, since all evaluated approaches converged toward solutions of comparable predictive quality. From a practical perspective, GA offers a favorable balance between accuracy and execution time for the evaluated defect density prediction setting.

Future work will investigate additional software engineering datasets, industrial repositories, multi-objective optimization techniques, AutoML strategies, and Agentic AI architectures capable of autonomously coordinating hyperparameter optimization processes.

Acknowledgments

This research was supported by National University of Costa Rica (No. 0127-23).

Declaration of Generative AI and AI-assisted Technologies in the Writing Process

The use of AI tools was limited to grammar, spelling, punctuation, and editorial revision. These tools were not used to generate experimental results or modify the scientific findings reported in this paper.

Appendix

Full tables are provided in the Supplementary Material [4].

References

1. Bashir, K., Li, T., Chubato, W.: An empirical study for enhanced software defect prediction using a learning-based framework. International Journal of Computational Intelligence Systems **12**, 282 (11 2018). https://doi.org/10.2991/ijcis.2018.125905638
2. Basili, V.R.: Software modeling and measurement: the goal/question/metric paradigm (1992)
3. Bischl, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., Thomas, J., Ullmann, L., Becker, J., Boulesteix, M., Deng, D., Lindauer, M.: Hyperparameter optimization: Foundations, algorithms, best practices and open challenges. arXiv preprint arXiv:2107.05847 (nov 2021)
4. Calvo, A.: Appendix: Extended tables for the optimml framework. Available in the GitHub repository "OptimML-Framework" (2025), access via GitHub: AndresACV/OptimML-Framework
5. Hesamolhokama, M., Shafiee, A., Ahmaditeshnizi, M., Fazli, M., Habibi, J.: Sdperl: A framework for software defect prediction using ensemble feature extraction and reinforcement learning (2024)
6. Huda, S., Alyahya, S., Ali, M.M., Ahmad, S., Abawajy, J., Al-Dossari, H., Yearwood, J.: A framework for software defect prediction and metric selection (1 2018)
7. Jakšić, Z., Devi, S., Jakšić, O., Guha, K.: A comprehensive review of bio-inspired optimization algorithms including applications in microelectronics and nanophotonics. Biomimetics **8**(3), 278 (Jun 2023). https://doi.org/10.3390/biomimetics8030278
8. Karaboga, D.: An idea based on honey bee swarm for numerical optimization. Tech. Rep. TR06, Erciyes University, Engineering Faculty, Computer Engineering Department (2005)
9. Kennedy, J., Eberhart, R.: Particle swarm optimization. In: Proceedings of the IEEE International Conference on Neural Networks. vol. 4, pp. 1942–1948 (1995). https://doi.org/10.1109/ICNN.1995.488968
10. Lamba, T., Kavita, Mishra, A.: Optimal metrics selection for software defect prediction. International Journal of Data Mining and Emerging Technologies **7**, 82 (1 2017). https://doi.org/10.5958/2249-3220.2017.00011.8
11. Lessmann, S., Baesens, B., Mues, C., Pietsch, S.: Benchmarking classification models for software defect prediction: A proposed framework and novel findings. IEEE Transactions on Software Engineering **34**(4), 485–496 (2008)

12. Malhotra, R.: Comparative analysis of statistical and machine learning methods for predicting faulty modules. Applied Soft Computing **21**, 286–297 (2014)
13. Menzies, T., Greenwald, J., Frank, A.: Data mining static code attributes to learn defect predictors. Software Engineering, IEEE Transactions on **33**(1), 2–13 (Jan 2007). https://doi.org/10.1109/TSE.2007.256941
14. Murillo, J., Quesada, C., Jenkins, M.: Software fault prediction: A systematic mapping study. In: XVIII Ibero-American Conference on Software Engineering. pp. 50–64. CibSE (2015)
15. Murillo-Morera, J., Castro-Herrera, C., Arroyo, J., Fuentes-Fernández, R.: An Empirical Validation of Learning Schemes using an Automated Genetic Defect Prediction Framework (July 2016), in press Springer
16. Murillo-Morera, J., Fuentes-Fernández, R., Calvo Vargas, A., Cambronero Delgado, A., Jadán-Guerrero, J., Arias, S.A.: A framework for hyperparameter configuration in the context of software defect density prediction using bio-inspired optimization algorithms. In: BioInspired Processing (BIP), 2025 IEEE 7th International Conference on (2025). https://doi.org/10.1109/BIP68491.2025.11489144
17. Murillo-Morera, J., Jenkins, M.: A software defect-proneness prediction framework: A new approach using genetic algorithms to generate learning schemes. In: The 27th International Conference on Software Engineering and Knowledge Engineering, SEKE 2015, Wyndham Pittsburgh University Center, Pittsburgh, PA, USA, July 6-8, 2015. pp. 445–450 (2015). https://doi.org/10.18293/SEKE2015-99
18. Murillo-Morera, J., Quesada-López, C., Castro-Herrera, C., Jenkins, M.: An empirical validation of an automated genetic software effort prediction framework using the isbsg dataset
19. Podgorelec, V.: Improved mining of software complexity data on evolutionary filtered training sets. WSEAS Trans. Info. Sci. and App **6**, 1751–1760 (2009)
20. Shepperd, M., Song, Q., Sun, Z., Mair, C.: Data quality: Some comments on the nasa software defect datasets. Software Engineering, IEEE Transactions on **39**(9), 1208–1215 (2013)
21. Singh, P., Verma, S.: Empirical investigation of fault prediction capability of object oriented metrics of open source software. In: Computer Science and Software Engineering (JCSSE), 2012 International Joint Conference on. pp. 323–327. IEEE (2012)
22. Song, Q., Jia, Z., Shepperd, M., Ying, S., Liu, J.: A general software defect-proneness prediction framework. Software Engineering, IEEE Transactions on **37**(3), 356–370 (2011)
23. Wang, H., Khoshgoftaar, T.M., Napolitano, A.: Software measurement data reduction using ensemble techniques. Neurocomputing **92**, 124–132 (2012)
24. Wang, Y., Cai, Z., Zhang, Q.: Differential evolution with composite trial vector generation strategies and control parameters. IEEE Transactions on Evolutionary Computation **15**(1), 55–66 (2011). https://doi.org/10.1109/TEVC.2010.2087271